

第 17 章 DNF 仓库、模块流与包组

从 repo 定义到可审计事务：把来源、元数据、候选、信任和状态放进同一条证据链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：DNF 不是看到 repo 文件后就直接安装包，而是依次读取来源定义、取得元数据、形成候选集合、验证信任、求解依赖并提交事务。

01仓库定义确认 repo ID、来源、启用和信任字段

02元数据与缓存确认 `repomd.xml` 和本地视图是否可用

03候选包解析结合架构、版本、过滤、stream 与优先级

04信任验证区分 TLS 传输链与 GPG 包签名链

05事务计划先读 Transaction Summary，再决定是否提交

06历史与交接用 history 审计，再转交 RPM、服务与功能层

专题地图

- 知识专题从仓库定义到事务：DNF 到底在解析什么
- 知识专题repo 文件、来源定位与信任字段
- 操作专题配置并分层验证一个自定义仓库
- 操作专题从需求定位候选软件并审阅事务
- 知识/操作模块 stream、profile 与模块过滤
- 知识专题包组、环境组、优先级与排除
- 操作专题建立本地仓库
- 诊断专题配置、元数据、TLS、GPG、过滤和依赖
- 经典任务配置可信仓库；诊断可见但不可用

阅读时持续回答

1. 当前证据是在证明配置、元数据、候选还是事务？
2. 仓库 ID、显示名和文件名分别是什么身份？
3. URL 是否真正指向包含 `repodata` 的仓库根？
4. 目标包为什么进入或离开当前候选集合？
5. TLS 与 GPG 分别验证哪一条信任链？
6. stream 与 profile 改变的是同一个状态吗？
7. Transaction Summary 会增加、升级或删除什么？
8. DNF 成功后还需要向哪一章交接？

阅读建议：每完成一个专题，都用“当前证据能证明什么、还不能证明什么、下一条证据是什么”复述一次；经典任务先独立完成，再对照参考解答检查证据链。

第 17 章 · 正文

一条 `dnf install` 命令背后并不只有“下载一个 RPM”。DNF 必须先读取客户端仓库定义，定位远端或本地内容根，取得并校验仓库元数据，再结合启用范围、架构、版本、模块流、优先级和排除规则形成候选集合。依赖求解器随后生成事务计划；只有事务真正提交后，RPM 已安装数据库才发生变化，DNF history 才留下可审计记录。

这条链上的每一层都可能独立失败。repo 文件存在，不代表仓库已被 DNF 正确识别；仓库出现在 `repolist`，不代表 `repomd.xml` 能够取得；元数据可用，不代表目标包没有被架构、exclude 或 module filtering 隐藏；候选包存在，不代表依赖一定可解；事务成功，也不代表应用配置、服务状态和外部功能已经正确。

- 仓库定义
 - 来源定位
 - 元数据与缓存
 - 候选包、模块和包组
 - TLS / GPG 信任
 - 依赖求解与事务
 - RPM 状态、history 与功能交接

本章围绕这条证据链组织内容。RPM 文件归属与校验留给第 16 章；网络、路由和 DNS 的完整配置留给第 18、19 章；本章只在诊断时识别这些层并给出交接证据。 `--allowerasing`、`gpgcheck=0` 和 `sslverify=0` 都不会被当成未经调查的默认答案。

概念

仓库定义 (repository definition) 是 `/etc/yum.repos.d/*.repo` 中以 `[repo-id]` 开始的客户端 section。它声明仓库身份、内容位置、默认启用状态和信任策略，却不保存软件包清单本身。repo 文件能被读取只证明配置层存在；只有后续元数据请求成功，才能继续讨论候选和事务。

概念

仓库元数据与缓存 (metadata and cache) 让 DNF 认识包名、版本、架构、依赖、文件、包组和可能存在的模块信息。仓库根通常以 `repodata/repomd.xml` 为入口，本地缓存只是客户端对该元数据的当前视图；清理缓存不能修复错误的 URL、DNS、服务器内容或 GPG key。

概念

GPG 信任 (GPG trust) 描述安装包是否由受信任签名者签发且内容未被篡改。`gpgcheck=1` 是校验策略，`gpgkey=` 是公钥入口；它们不能互相替代。HTTPS/TLS 保护传输通道和服务器身份，GPG 保护包签名，两条信任链必须根据错误层分别调查。

概念

候选包 (package candidate) 是当前仓库集合、架构、版本、优先级、排除规则和模块状态共同允许求解器选择的软件包构建。服务器目录中存在某个 RPM，并不意味着它一定处于客户端当前候选集合；`repoquery` 观察的正是这个被规则塑造后的视图。

概念

模块流 (module stream) 表示同一软件栈的一条版本线。启用 stream 会改变模块化包的候选集合，但不等于已经安装其中的全部内容；流名称、默认标记和可用版本必须从当前仓库元数据读取，不能把另一台机器或另一小版本的输出当成固定事实。

概念

profile 是模块针对 server、client、development 等用例定义的软件包集合。`module enable NAME:STREAM` 选择版本线，`module install NAME:STREAM/PROFILE` 才按用例提交安装事务；stream 是版本状态，profile 是安装选择，两者不能混为一谈。

概念

包组与环境组 (package group / environment group) 是仓库元数据描述的软件集合，不是一个 RPM。成员可能分为 mandatory、default、optional 和 conditional；组安装只说明一组包按元数据进入事务，不保证其中的服务已经配置、启动或能够对外提供功能。

概念

事务历史 (transaction history) 是 DNF 对已提交软件事务的审计记录，能够回答某次操作安装、升级或删除了哪些包。history 不是无条件回滚系统的快照：`undo` 或 `rollback` 仍依赖旧构建可获得、依赖关系可满足；Red Hat 不支持借此把关键 RHEL 系统包降级到旧版本，因此它们不能被当成核心系统包的盲目恢复按钮。

操作语义

先理解命令作用对象，再记关键形式。速查区只覆盖本章最重要的入口；正文专题仍负责解释状态、证据、验证和边界。

dnf repolist / makecache / repoquery

SYNOPSIS

```
<pre><code>dnf repolist [--all]\ndnf makecache [--refresh]\ndnf repoquery [OPTIONS]
[PACKAGE ...]</code></pre>
<p>依次观察仓库是否被识别、真实刷新元数据、查询当前候选内容。三条命令证明的层次不同，不能互相替代。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`repolist --all`</dt><dd>列出启用和禁用仓库，验证配置解析与默认启用状态。</dd>
<dt>`makecache --refresh`</dt><dd>强制重新检查启用仓库元数据，暴露 URL、TLS、HTTP 和元数据错误。
</dd><dt>`repoquery --repo ID PKG`</dt><dd>把候选查询限制到指定 repo ID，证明该仓库提供目标构建。
</dd><dt>`--installed` / `--available`</dt><dd>分别观察已安装集合或当前可用候选集合。</dd></dl>
```

repo 配置文件

SYNOPSIS

```
<pre><code>/etc/yum.repos.d/NAME.repo\n[repo-id]\nname=...\nbaseurl=... | mirrorlist=...
| metalink=...\nenabled=0|1\ngpgcheck=0|1\ngpgkey=...</code></pre>
<p>持久定义仓库身份、来源、默认参与状态和包签名策略。文件名、repo ID 与显示名称是不同身份。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`baseurl`</dt><dd>直接指向包含 `repodata` 的仓库根。</dd><dt>`mirrorlist`</dt><dd>指向返回可用镜像 URL 列表的服务。</dd><dt>`metalink`</dt><dd>提供镜像选择及附加元数据。</dd>
<dt>`enabled`</dt><dd>控制仓库是否默认参与普通查询和求解。</dd><dt>`gpgcheck` / `gpgkey`</dt>
<dd>分别声明包签名校验策略和公钥入口。</dd></dl>
```

dnf install / upgrade / remove / history

SYNOPSIS

```
<pre><code>dnf install [OPTIONS] PACKAGE ...\ndnf upgrade [PACKAGE ...]\ndnf remove
PACKAGE ...\ndnf history [list|info|undo|rollback] [ID]</code></pre>
<p>让求解器生成并提交软件事务，再通过 history 审计已发生的变化。删除和恢复类操作必须先读完整事务摘要。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`--assumeno`</dt><dd>生成事务计划但默认拒绝提交，适合预览安装、升级或删除影响。</dd><dt>`--
enablerepo=ID`</dt><dd>只在当前命令临时启用指定仓库，不改变 repo 文件。</dd><dt>`--
disablerepo=PATTERN`</dt><dd>只在当前命令临时缩小仓库范围；限定来源时必须保留依赖仓库。</dd>
<dt>`history info ID`</dt><dd>读取指定事务的命令、状态和包变化。</dd></dl>
```

dnf module

SYNOPSIS

```
<pre><code>dnf module list [NAME]\ndnf module info NAME[:STREAM]\ndnf module enable
NAME:STREAM\ndnf module install NAME:STREAM[/PROFILE]\ndnf module reset|disable
NAME</code></pre>
<p>观察并改变模块版本线和 profile 安装选择。启用 stream 与安装 profile 改变的是不同状态。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`NAME:STREAM`</dt><dd>显式选择模块及版本线，避免依赖模糊默认值。</dd>
<dt>`NAME:STREAM/PROFILE`</dt><dd>按指定用例安装 profile 中的软件集合。</dd><dt>`reset`</dt>
<dd>清除显式 stream 选择，不等同于删除已安装包。</dd><dt>`disable`</dt><dd>使该模块的流不参与普通模
块化候选解析。</dd></dl>
```

dnf group

SYNOPSIS

```
<pre><code>dnf group list [--hidden]\ndnf group info GROUP\ndnf group install GROUP\ndnf
group remove GROUP</code></pre>
<p>查询和提交包组或环境组事务。先用 `group info` 读取 mandatory、default 和 optional 成员，再决定
安装范围。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`list --hidden`</dt><dd>同时查看默认隐藏的组。</dd><dt>`group info`</dt><dd>读取组
ID、描述和成员分类。</dd><dt>`group install`</dt><dd>按组元数据形成安装事务，不等于完成服务配置。
</dd></dl>
```

createrepo_c

SYNOPSIS

```
<pre><code>createrepo_c [OPTIONS] DIRECTORY</code></pre>
<p>扫描普通 RPM 目录并生成可供 DNF 读取的 `repodata`。它不会凭空恢复缺失的包组或模块语义。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`DIRECTORY`</dt><dd>包含目标 RPM 的仓库内容目录。</dd><dt>`--update`</dt><dd>在已有元数
据基础上更新普通仓库索引。</dd><dt>`file:///PATH`</dt><dd>客户端 repo 定义引用本地绝对路径时使用三
条斜线。</dd></dl>
```

[知识专题] 从仓库定义到事务：DNF 到底在解析什么

排查软件安装问题时，最容易犯的错误是把“repo 文件存在”“仓库已启用”“元数据能下载”“包能查询”“依赖能求解”和“事务已完成”混成一个状态。它们分别属于不同对象，也需要不同证据。先把对象分开，才能知道失败停在哪一层。

① [知识点] 仓库定义、仓库内容和本地缓存是三个对象

repo 文件只是客户端配置。它描述仓库 ID、来源、启用和信任策略，但不包含真实的软件包清单。远端或本地仓库内容由 `repodata` 和 RPM 构成；客户端还会在本地保存一份元数据缓存，以便后续查询和求解。

/etc/yum.repos.d/exam.repo	客户端持久定义
http://content/.../BaseOS/	仓库内容根
/var/cache/dnf/...	客户端元数据和下载缓存

修改 repo 文件后，旧缓存可能仍存在。相反，删除缓存不会修复错误的 URL、DNS 或 GPG key。诊断时必须确认自己正在检查哪一层。

② [知识点] repomd.xml 是仓库元数据的入口，不是包本身

典型仓库根下存在：

```
repopdata/repomd.xml
Packages/ 或其他 RPM 路径
```

repomd.xml 记录其他元数据文件的位置和校验值。DNF 获取它之后，才能继续读取包名称、版本、架构、依赖、文件列表、包组或模块等信息。错误 URL 常表现为下载 repodata/repomd.xml 时返回 404；这通常说明 baseurl 没有指向真正的仓库根。

③ [知识点] 候选集合由多层规则共同缩小

仓库中存在某个包，只说明服务端保存了它。要进入当前 DNF 候选集合，还要满足：

- 仓库被当前命令启用；
- 架构适配本机或属于允许的 noarch；
- 版本没有被排除规则过滤；
- module filtering 没有隐藏它；
- 优先级和重复候选选择规则允许它参与；
- 必要的依赖也能从当前仓库集合中获得。

因此“包明明在服务器上”不能直接推出客户端一定能安装。

④ [知识点] 查询、求解和提交事务是三个阶段

dnf repoquery 主要回答“当前元数据中有哪些候选及其关系”；dnf install --assumeno 可让求解器生成事务摘要而不提交；真正执行 dnf install 后，才改变 RPM 已安装数据库并产生 DNF history 记录。

```
repoquery 有结果
≠ 依赖一定可解
≠ 事务已执行
≠ 应用已完成配置
```

已安装数据库、文件归属和 RPM 校验属于第 16 章；服务的 active/enabled 和功能属于相应服务章节。本章只建立必要接口。

⑤ [知识点] DNF 成功只覆盖软件事务层

DNF 返回成功可以证明求解和包事务没有报告失败，但不能证明：

- 配置文件符合业务要求；
- systemd 服务正在运行或开机启动；
- 网络端口已监听；
- 防火墙与 SELinux 已允许访问；
- 用户从外部获得了正确功能。

考试题若最终要求“服务可用”，DNF 事务只是中间证据，不是最终终态。

[Cheatsheet] repo 文件是定义；`repopdata` 是仓库元数据；缓存是客户端视图；`repolist` 看配置，`makecache` 取元数据，`repoquery` 看候选，事务改变已安装状态；任何一层成功都不能替代下一层验证。

[知识专题] repo 文件、来源定位与信任字段

repo 文件看起来只是几行 INI 配置，但每个字段回答不同问题：仓库叫什么、从哪里取内容、是否参与当前求解、是否校验包签名、使用哪把公钥。最稳妥的切入方式是先分清“身份”“位置”“启用”和“信任”，再讨论临时覆盖和变量展开。

① [知识点] section ID、文件名和 `name` 不是同一身份

```
[exam-baseos]
name=Exam BaseOS for RHEL 9
```

- `exam-baseos` 是仓库 ID，供 `--repo`、`--enablerepo` 等选项引用；
- `name` 是面向人的显示名称；
- `exam.repo` 只是承载配置的文件名。

仓库 ID 应在全部已加载配置中唯一。两个文件重复定义同一 ID 会制造覆盖或重复定义风险，不能仅靠文件名判断实际加载结果。

② [知识点] `baseurl`、`mirrorlist` 和 `metalink` 是三种定位方式

`baseurl` 直接给出一个或多个仓库根；`mirrorlist` 指向返回镜像 URL 列表的服务；`metalink` 除镜像信息外还可带有用于选择和完整性检查的附加元数据。一个仓库通常选择一种主要定位方式，避免同时留下互相矛盾的来源。

```
baseurl=https://repo.example.com/rhel9/BaseOS/
# 或
mirrorlist=https://mirror.example.com/rhel9/baseos.list
# 或
metalink=https://mirrors.example.com/metalink?repo=rhel-9-baseos
```

本章不要求搭建 mirrorlist 或 metalink 服务，但要求能够识别字段、读取最终错误 URL，并知道 `baseurl` 必须指向包含 `repodata` 的内容根。

③ [知识点] `DNF` 变量与 `Shell` 变量属于不同解析器

repo 配置可使用 `$releasever`、`$basearch` 等 `DNF` 变量。例如：

```
baseurl=https://repo.example.com/rhel/$releasever/$basearch/BaseOS/
```

若用未引用的 here-document 在 `Shell` 中生成文件，`Shell` 可能先尝试展开 `$releasever`，导致空字符串或错误路径。可以使用单引号保护的 here-document：

```
cat > /etc/yum.repos.d/exam.repo <<'EOF'
[exam-baseos]
name=Exam BaseOS
baseurl=https://repo.example.com/rhel/$releasever/$basearch/BaseOS/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
EOF
```

题目给出固定 `RHEL 9` URL 时，直接使用题目参数，不自作主张替换成别的小版本或 `RHEL 10` 路径。

④ [知识点] `enabled` 只控制是否参与普通命令

`enabled=1` 表示仓库默认参与查询和求解；`enabled=0` 表示默认禁用。命令级选项可以临时覆盖：

```
dnf --enablerepo=exam-appstream repoquery httpd
dnf --disablerepo='*' --enablerepo=exam-baseos,exam-appstream install httpd
```

这种覆盖只影响当前命令，不修改 repo 文件。第二种“只启用指定仓库”只有在题目明确要求限定来源时才使用，并且必须把依赖仓库一起纳入，否则会人为制造依赖缺口。

⑤ [知识点] `gpgcheck` 与 `gpgkey` 分别描述策略和密钥入口

`gpgcheck=1` 要求安装包通过 GPG 签名检查；`gpgkey=` 指向可用于导入或取得公钥的位置。二者不是同义字段：配置了 key 但关闭 `gpgcheck`，仍然没有执行目标校验；开启校验但 key 错误，则事务会失败。

```
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
```

GPG 失败的正确方向是核对仓库来源、key 文件、签名者和完整错误，而不是直接改为 `gpgcheck=0`。

⑥ [知识点] TLS 信任与 GPG 信任是两条不同链

HTTPS/TLS 主要保护传输通道和服务器身份；GPG 包签名用于验证软件包是否由受信任签名者签发、内容是否被篡改。可能出现：

- TLS 失败，尚未取得元数据；
- TLS 成功，但 GPG key 不匹配；
- 两者都成功，但包被模块或排除规则过滤。

排错时根据实际错误层推进，不把一种信任机制替代另一种。

⑦ [边界] `sslverify=0` 和 `gpgcheck=0` 都不是默认修复

关闭 TLS 证书验证或包签名校验会移除安全属性，只能掩盖错误，不能证明来源正确。考试与真实工作中应优先修复：

错误的 URL / DNS / 时间 / CA 信任
错误或缺失的 GPG key
仓库内容与 key 不匹配

[Cheatsheet] `[repo-id]` 是命令行身份，`name` 是显示名；三种来源字段为 `baseurl/mirrorlist/metalink`；`enabled` 控制默认参与；`gpgcheck` 是校验策略，`gpgkey` 是密钥入口；TLS 与 GPG 分层验证，不靠关闭保护通过任务。

[操作专题] 配置并分层验证一个自定义仓库

仓库配置操作的正确顺序不是“写完文件就安装”，而是先建立基线，再完成最小定义，随后依次验证解析、元数据和代表包。这样一旦失败，可以准确知道问题位于配置、传输、元数据还是内容层。

① [操作] 变更前记录现有仓库基线

作用对象：当前 DNF 配置和启用仓库集合。基本语义：在写入新 repo 前保存能解释后续变化的只读证据。典型形式：

```
ls -l /etc/yum.repos.d/  
dnf repolist --all  
dnf module list --enabled
```

如果系统属于订阅环境，还应避免随意删除现有 Red Hat 仓库定义。本章不展开注册和订阅管理，但要求不以“清空目录”作为配置自定义仓库的默认步骤。

② [操作] 写入最小且可审计的 repo section

```
[exam-baseos]
name=Exam BaseOS
baseurl=http://content.example.com/rhel9.0/x86_64/dvd/BaseOS
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release

[exam-appstream]
name=Exam AppStream
baseurl=http://content.example.com/rhel9.0/x86_64/dvd/AppStream
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
```

关键参数：题目给出的 ID、URL 和 key 必须逐项保留；BaseOS 与 AppStream 常互相补充，不能仅凭目标包名称猜只需一个仓库。验证与边界：静态阅读配置可以发现拼写、重复 ID 和明显路径错误，但不能证明服务器可达。

③ [操作] 使用 `dnf repolist --all` 验证解析和启用状态

```
dnf repolist --all
```

这一步能确认：

- repo 文件语法至少能够被 DNF 读取；
- 仓库 ID 出现在加载结果中；
- 当前持久启用状态是 enabled 还是 disabled。

它不能充分确认 URL 可达，也不能证明 repodata 完整。看到仓库 ID 后仍要继续刷新元数据。

④ [操作] 使用 `makecache --refresh` 真实请求元数据

```
dnf makecache --refresh
```

`--refresh` 让 DNF 把元数据视为过期并重新检查。该命令能够暴露：

- DNS 或连接错误；
- TLS/CA 错误；
- HTTP 403、404、5xx；
- 错误仓库根；
- `repomd.xml` 或元数据校验失败。

不要在不知道错误类型时反复执行 `dnf clean all`。清缓存只是移除本地状态；如果错误在 URL 或服务器端，清理不会产生修复效果。

⑤ [操作] 用代表包证明仓库内容符合预期

```
dnf repoquery --repo exam-baseos bash
dnf repoquery --repo exam-appstream httpd
```

作用对象：指定仓库的候选包元数据。基本语义：把查询范围限制到某个 repo ID，确认该仓库确实提供预期内容。边界：一个代表包有结果只能证明该查询成功，不能证明仓库全部 RPM、组或模块元数据都完整。

⑥ [验证] 建立四层证据矩阵

层	推荐证据	证明范围
配置	读取 <code>.repo</code> 、 <code>dnf repolist --all</code>	ID、字段和启用状态
元数据	<code>dnf makecache --refresh</code>	仓库元数据可取得并通过校验
内容	<code>dnf repoquery --repo ... <pkg></code>	指定仓库提供代表候选
事务	<code>dnf install</code> 、 <code>dnf history info</code>	求解并提交了软件包事务

四层都通过后，才可以说“仓库能够支撑这个安装任务”；仍不能把它扩大为服务功能已经正确。

[Cheatsheet] 基线 `repolist --all` → 写最小 repo → 再次 `repolist --all` → `makecache --refresh` → `repoquery --repo` → 事务。不要清空所有仓库，不关闭 GPG，不把仓库可见等同于可用。

[操作专题] 用 DNF 和 repoquery 从需求定位候选软件

软件题经常只给“需要某个命令”“需要一个 Web 服务器”或“某文件应由哪个包提供”，不直接给包名。此时要先判断已知信息属于描述、包名、命令路径还是依赖关系，再选择最窄的查询接口。查询结果是证据，不应机械安装第一项。

① [操作] 已知描述时使用 `dnf search`

```
dnf search 'web server'
dnf search --all 'network time'
```

`search` 匹配包名称和摘要；`--all` 扩大到描述等字段。结果可能包含工具、库、文档和兼容包，应继续使用 `dnf info` 或 `repoquery` 确认候选身份。

② [操作] 已知包名时区分 installed 和 available

```
dnf list --installed httpd
dnf list --available httpd
dnf info httpd
```

`dnf info` 可能同时出现 installed 与 available 区块。要回答“本机是否已经安装”，最终仍应引用第 16 章的 RPM 查询；要回答“仓库提供什么版本”，使用 available 或 repoquery。

③ [操作] 已知命令或路径时使用 `dnf provides`

```
dnf provides '*/semanage'
dnf provides /usr/bin/rsync
```

通配模式必须引用，避免 Shell 在当前目录先展开。查询结果可能包含旧版本、不同架构或兼容子包，应结合目标路径、RHEL 9 仓库和架构选择。

④ [操作] 使用 repoquery 查看来源、版本和架构

```
dnf repoquery --available --qf '%{name}-%{epoch}:%{version}-%{release}.*{arch} %{repoid}'
httpd
dnf repoquery --repo exam-appstream httpd
```

自定义查询格式可以同时显示候选标识和来源仓库。实际可用的格式字段应以当前 `dnf repoquery --querytags` 或帮助为准；讲义中的骨架用于说明查询方向，不虚构固定输出。

⑤ [操作] 查询包文件与依赖关系

```
dnf repoquery -l httpd
dnf repoquery --requires httpd
dnf repoquery --whatrequires 'libssl.so.3()(64bit)'
```

- `-l` 读取仓库包的文件列表，不要求包已安装；
- `--requires` 读取包声明的依赖；
- `--whatrequires` 查找声明依赖某能力的候选。

依赖查询用于解释求解错误，但不能取代完整事务摘要，因为弱依赖、冲突、模块和已安装状态也会影响最终计划。

⑥ [诊断] 无结果时按过滤链排查

```
包名或路径是否正确
→ 当前启用仓库是否包含目标内容
→ 架构是否匹配
→ 是否被 excludepkgs/includepkgs 过滤
→ 是否受到 module filtering
→ 元数据缓存是否对应当前配置
```

不要在第一步就使用 `--allowerasing`。该选项处理的是冲突事务中允许删除包的问题，不能修复仓库缺失、拼写错误或模块候选不可见。

[Cheatsheet] 描述用 `search`；包名用 `list/info`；命令或文件用 `provides`；来源、版本、文件、依赖用 `repoquery`。无结果依次查 repo、架构、exclude、module 和缓存。

[操作专题] 安装、升级、删除与 DNF 历史

DNF 改变系统前会先生成事务计划。稳定操作的关键不是快速输入 `y`，而是读懂将安装、升级、降级或删除哪些包，以及这些变化来自哪个仓库。事务执行后再用 `history` 和 `RPM` 层证据确认结果。

① [操作] 安装与重新安装

```
dnf install httpd
dnf reinstall httpd
```

`install` 在候选集合中选择满足条件的包并解决依赖；`reinstall` 重新安装当前版本或可用等价构建，适用于已确认包文件损坏且配置边界清楚的情况。它不是处理未知应用故障的第一步，因为配置文件、数据和服务状态可能不由重装自动恢复。

② [操作] 检查和执行升级

```
dnf check-upgrade
dnf upgrade httpd
dnf upgrade
```

- `check-upgrade` 只检查可用更新；其退出状态和输出应按帮助页解释；
- 指定包升级能限制影响范围；
- 无参数 `upgrade` 可能改变大量系统包，考试题未要求全量更新时不要扩大范围。

升级前应确认仓库集合和候选来源，避免临时仓库意外替换已有包。

③ [操作] 删除前必须阅读 Transaction Summary

```
dnf remove httpd
```

删除事务可能同时移除依赖或不再需要的软件包。提交前至少检查：

- 目标包是否正确；
- 是否出现不期望的服务、库或工具；
- 是否会破坏其他题目已完成的功能；
- 删除数量和下载/释放空间是否符合预期。

包名不是实际影响集合。若摘要异常，应取消并调查依赖、模块和组状态。

④ [操作] 使用 `--assumeno` 预览事务

```
dnf install --assumeno httpd
dnf remove --assumeno httpd
```

该选项允许 DNF 进行查询和求解，但在确认阶段默认回答否。它适合建立变更前证据。它仍可能刷新元数据，因此不是绝对“零外部访问”；但不会提交包安装或删除事务。

⑤ [操作] 使用 history 审计事务

```
dnf history
dnf history info 12
```

`history` 列出事务 ID、时间、动作和结果摘要；`history info` 展开指定事务涉及的软件包和命令行。它可以帮助回答“刚才改变了什么”，但不是完整业务审计系统，也不能证明包外配置未被修改。

⑥ [边界] `undo` 和 `rollback` 不是无条件恢复按钮

```
dnf history undo 12
dnf history rollback 10
```

这些操作仍需重新求解事务，并依赖旧版本在当前仓库中可取得。若历史事务涉及核心系统包、模块流、后来安装的软件或已删除仓库，恢复可能失败或产生额外影响。执行前先预览摘要并确认回退目标；考试中没有明确要求时，不把 `rollback` 当成首选补救。

⑦ [验证] 从事务交接到已安装状态和功能

```
DNF 事务摘要和 history
→ rpm -q / rpm -qf (第 16 章)
→ 配置文件与语法
→ systemd 当前/持久状态 (第 12 章)
→ 监听、访问或数据结果
```

本章讲清交接点，但不重复相邻章节的完整操作。

[Cheatsheet] 安装 `install`，修复已确认包内容可考虑 `reinstall`，升级先 `check-upgrade`，删除前读完整摘要，预览用 `--assumeno`，历史用 `history/info`；undo/rollback 仍受仓库和依赖约束。

[知识专题] 模块、stream、profile 与模块过滤

模块解决的是“同一软件栈存在不同生命周期或版本线时，仓库如何提供可控选择”。它在普通包元数据之上增加 stream 和 profile 状态，因此模块问题不能只用包名解释。最顺的学习路径是先识别四个对象，再理解启用、安装、重置和禁用分别改变什么。

① [知识点] module、stream、profile 和 RPM 是四个层次

```
module: nginx
stream: 1.22
profile: common / server / development
packages: nginx、依赖库、工具等 RPM
```

具体名称、流和 profile 取决于当前 RHEL 9 仓库元数据。讲义不要求背诵某个流一定存在，而要求先查询后操作。

② [知识点] stream 表示版本线，不只是一个包版本

启用一个 stream 会让该 stream 的模块化包参与求解，并可能过滤其他流或非模块候选。一个 stream 可以随仓库更新得到新的构建，但仍属于同一版本线。不能把 stream 当成单个 RPM 的完整 NEVRA。

③ [知识点] profile 是安装用例，不是模块状态本身

profile 描述某个使用场景所需的包集合。例如 common、server、client、development 可能包含不同成员。启用 stream 后不一定安装任何包；安装 profile 才会提交对应软件包事务。

④ [知识点] 模块状态需要从真实输出读取

```
dnf module list
dnf module list --enabled
dnf module list --installed
dnf module info <module>
```

输出标记和可用流会随小版本、仓库和语言环境变化。学习时关注：哪些流可用、哪个流当前 enabled、是否有 installed profile，而不是背固定截图。

⑤ [知识点] `enable` 与 `install` 改变不同对象

```
dnf module enable <module>:<stream>
dnf module install <module>:<stream>/<profile>
```

- `enable` 选择活动 stream，改变候选集合；
- `install` 在选择 stream 的同时安装 profile 或默认 profile 中的包。

题目只要求版本流选择时，不额外安装 profile；题目只要求普通包且未提模块时，先查询候选，不主动改变模块状态。

⑥ [知识点] `reset`、`disable` 和 `remove` 不是同义操作

- `module reset` 清除显式 stream 选择，通常不自动删除已安装包；
- `module disable` 阻止该模块各 stream 作为模块候选；
- `module remove` 计划移除模块 profile 的软件内容，实际影响必须读事务摘要。

重置状态后，现有已安装包仍可能存在。要回答“系统还装着什么”，应转到 RPM 层查询。

⑦ [边界] 切换 stream 是有状态迁移

在支持的环境中，`dnf module switch-to` 可用于从一个 stream 迁移到另一个 stream。实际事务取决于当前安装内容和仓库可用构建。切换前应：

```
记录当前 enabled/installed 状态
→ 查看目标 stream 和 profile
→ 预览事务摘要
→ 确认是否降级、升级或删除包
→ 提交后重新验证模块和已安装状态
```

不把 `reset + enable` 的随意组合当成通用切流答案。

⑧ [诊断] `module filtering` 会制造“仓库有包但查询不到”

当某包存在于仓库但被非活动 stream 隐藏时，普通 `dnf list` 或安装可能显示没有匹配项或被过滤。此时最有区分度的证据是：

```
dnf module list
dnf module provides '<package-or-path>'
dnf repoquery --available <package>
```

结合完整 DNF 错误判断是模块过滤、排除规则还是仓库缺失。

[Cheatsheet] module 是软件栈，stream 是版本线，profile 是用例包集合，RPM 是实际安装单元；enable 选流，install 装 profile，reset 不等于卸载，disable 阻止模块流，切流必须读事务影响。

[操作专题] 查询和改变模块状态

模块操作不应从 enable 开始，而应从“当前仓库实际提供什么”开始。下面的顺序把查询、选择、安装和验证分开，避免在不知道当前状态时盲目 reset 或切流。

① [操作] 查询可用流和 profile

```
dnf module list <module>
dnf module info <module>:<stream>
dnf module info --profile <module>:<stream>
```

验证：确认题目指定的 stream 和 profile 确实存在。若不存在，先检查 AppStream 元数据、RHEL 小版本和仓库来源，而不是创建一个近似名称。

② [操作] 只启用指定 stream

```
dnf module enable <module>:<stream>
```

提交前阅读事务摘要或模块状态变化。操作后：

```
dnf module list --enabled <module>
```

这只能证明流选择，不证明任何 profile 已安装。

③ [操作] 安装显式 profile

```
dnf module install <module>:<stream>/<profile>
```

省略 profile 时可能安装默认 profile；考试需要精确内容时应使用题目给出的 profile。完成后同时检查：

```
dnf module list --installed <module>
rpm -q <representative-package>
```

第二条属于第 16 章接口，用来证明实际包状态。

④ [操作] 重置或禁用前先建立基线

```
dnf module list --enabled
dnf module list --installed
dnf module reset <module>
# 或按明确目标:
dnf module disable <module>
```

`reset` 后重新读取 module list，并单独检查已安装包是否仍存在。`disable` 可能影响后续更新和依赖求解，不能作为解决普通包冲突的随手操作。

⑤ [操作] 切流前使用预览和分层验证

```
dnf --assumeno module switch-to <module>:<target-stream>
```

若当前工具支持该组合，先查看拟升级、降级、替换或删除的包；真实命令语法以 `dnf module switch-to --help` 为准。提交后分别验证：目标 stream、installed profile、代表包版本和应用功能。

[Cheatsheet] 先 `module list/info`，再按题意 enable 或 install；显式 profile 更可控；reset/disable 前后分开查 module 与 RPM；切流必须预览事务，不背固定流。

[知识专题] 包组、环境组与成员分类

包组用于一次表达一类系统能力，例如开发工具或服务器环境。它的价值是让仓库维护者定义“这个用例通常需要哪些包”，但组不是不可分割的整体。理解成员分类和 DNF 的组状态，才能解释为什么某些包已安装、某些 optional 包没有安装，或者组删除后仍有成员保留。

① [知识点] 包组和环境组来自仓库元数据

- package group：一组相关软件包；
- environment group：可以包含多个包组和额外包，表达更大的系统环境。

如果仓库没有相应 comps/group 元数据，`dnf group list` 就无法凭 RPM 文件名自动推断组。

② [知识点] mandatory、default、optional 和 conditional 含义不同

- mandatory：组的必要成员；
- default：正常组安装时默认选择；
- optional：只有显式请求或相关选项时才安装；
- conditional：在满足关联条件时加入。

查看组详情比背“组会安装哪些包”更可靠：

```
dnf group info 'Development Tools'
```

③ [知识点] 显示名和 group ID 的稳定性不同

显示名可能受语言环境影响，脚本和考试操作更适合先通过 `group list --hidden -v` 或 `group info` 找到稳定 ID。不要因中文或英文名称不同误判组不存在。

④ [操作] 查询、安装和删除组

```
dnf group list
dnf group list --hidden
dnf group info '<group-name-or-id>'
dnf group install '<group-name-or-id>'
dnf group remove '<group-name-or-id>'
```

删除组前仍要读 Transaction Summary。用户后来显式安装、其他组共享或其他软件依赖的成员，可能不会简单按“组内全部包”删除。

⑤ [边界] 组安装标记与包实际存在是两个维度

一组成员包可能在安装组之前已经存在，DNF 还会记录组事务和安装原因。反过来，手工安装部分成员不一定让 DNF 认为整个组已安装。验证时既看 group 状态，也按题目要求检查代表包。

⑥ [边界] 包组只完成软件集合，不完成服务配置

安装“Web Server”或类似组后，相关服务仍可能：

- 未启用或未启动；
- 使用默认配置；
- 被 firewalld 或 SELinux 阻止；
- 缺少题目要求的数据。

组事务必须交接到对应服务和安全章节继续验收。

[Cheatsheet] 组来自仓库 comps 元数据；成员分 mandatory/default/optional/conditional；先 `group list/info`，再 `install/remove`；显示名可能本地化，ID 更稳定；组安装不等于服务可用。

[知识专题] 多仓库优先级、成本与排除规则

多个仓库提供同名包时，DNF 不只是“任选一个最高版本”。仓库优先级、成本、包排除、命令级启用范围和模块过滤都会影响候选。这里覆盖 RHCSA 和常见工作所需的必要范围，不扩展为完整企业内容治理策略。

① [知识点] `priority` 数值越小，仓库优先级越高

```
priority=10
```

当不同优先级仓库提供同名候选时，高优先级仓库可阻止低优先级仓库的更高版本被选中。默认值和精确比较细节应以 RHEL 9 的 `dnf.conf(5)` 为准；操作时关注“数值越小越优先”以及它可能导致版本看似被压低。

② [知识点] `cost` 用于同优先级来源的偏好

较低 `cost` 通常更受偏好，可用于倾向本地或较便宜的镜像。它不等同于 `package priority`，也不能替代 GPG、版本或模块规则。考试不要求时不随意增加 `cost`。

③ [知识点] `excludepkgs` 和 `includepkgs` 改变候选集合

```
excludepkgs=kernel* example-old*  
includepkgs=httpd* mod_ssl
```

排除可以出现在全局或仓库级配置。一个包“明明能在仓库服务器看到但 DNF 不显示”，应检查 `exclude`。
`includepkgs` 则把仓库内容限制到匹配集合。

④ [操作] 使用 `--disableexcludes` 进行有意识的诊断覆盖

```
dnf --disableexcludes=<repoid> repoquery <package>
```

该选项适合验证“是否确实被 `exclude` 隐藏”。它不是持久修复，也不应无条件用于安装；确认原因后，应修正错误的排除策略或按题目要求保留策略。

⑤ [知识点] 命令级 `repo` 范围可以比持久配置更窄

`--disablerepo='*' --enablerepo=...` 会改变当前命令的仓库集合。若遗漏 BaseOS 或依赖仓库，可能出现“目标包存在但依赖不可解”。因此限定来源时必须把“目标包仓库”和“依赖仓库”一起考虑。

⑥ [诊断] 优先级、排除和模块过滤要分别取证

```
repoquery --repo <id> 能看到包吗？  
→ 普通 repoquery 能看到吗？  
→ --disableexcludes 后是否出现？  
→ module list/provides 是否显示过滤关系？  
→ 事务摘要最终选择哪个 repoid？
```

每一步只改变一个变量，避免同时启用所有仓库、禁用排除和 `reset` 模块，导致无法解释真正原因。

[Cheatsheet] `priority` 小者优先，`cost` 用于同层来源偏好；`exclude/include` 改变候选；临时 `--disableexcludes` 只用于证明原因；限定仓库时别遗漏依赖仓库；与 `module filtering` 分开调查。

[操作专题] 从 RPM 目录或安装介质建立本地仓库

本地仓库常用于无外网实验、受控内容分发或离线安装。它的本质仍是“客户端能通过仓库协议读取元数据和包”，而不是“文件就在磁盘上”。先判断目录是否已有 `repodata`，再决定直接配置还是使用 `createrepo_c`。

① [知识点] 安装介质与普通 RPM 目录不同

RHEL 安装介质中的 BaseOS/AppStream 通常已经带有 `repodata`，挂载后可直接作为 `file://` 仓库来源。普通收集目录只有 RPM 时，需要生成元数据。

```
find /mnt/rhel9 -maxdepth 3 -name repomd.xml -print
```

看到 `repodata/repomd.xml` 才说明目录具备普通仓库入口。

② [操作] 为普通 RPM 目录生成元数据

```
mkdir -p /srv/repo/custom
cp /path/to/packages/*.rpm /srv/repo/custom/
createrepo_c /srv/repo/custom
```

作用对象：目标目录中的 RPM 集合。基本语义：扫描软件包并生成 `repodata`。验证：

```
test -f /srv/repo/custom/repodata/repomd.xml
```

命令成功和文件存在只能证明普通元数据入口生成，不能证明每个 RPM 签名、依赖或高级元数据都满足任务。

③ [操作] 内容变化后更新仓库元数据

```
cp new-package.rpm /srv/repo/custom/
createrepo_c --update /srv/repo/custom
```

删除或替换 RPM 后也要重新生成或更新元数据。客户端随后使用：

```
dnf clean metadata
dnf makecache --refresh
```

只在确有旧元数据问题时清理相关缓存，不机械执行 `clean all`。

④ [操作] 配置 `file://` 本地仓库

```
[local-custom]
name=Local Custom Repository
baseurl=file:///srv/repo/custom
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-example
```

`file:///` 中三个斜杠表示本机绝对路径。DNF 进程需要对路径和文件具有遍历与读取权限。SELinux 或挂载状态问题若影响读取，应转交相应章节，不以 `chmod 777` 作为默认答案。

⑤ [边界] 本地文件不等于可信文件

内容位于本机磁盘并不自动证明来源可信。若题目要求 GPG 校验，应提供与包签名匹配的 key；自己制作的未签名包仓库需要明确的信任与签名流程，本章不以关闭校验来回避。

⑥ [边界] 普通 `createrepo_c` 不自动重建模块语义

模块仓库需要模块元数据。仅把模块 RPM 拷贝到目录并运行普通 `createrepo_c`，可能得到可读取的普通包元数据，却失去 `stream/profile` 和 `module filtering` 语义。复制或同步模块内容时应保留/注入相应模块元数据；该高级制作流程只建立边界，不作为 RHCSA 必做操作。

⑦ [验证] 从目录推进到客户端候选

```
test -f /srv/repo/custom/repodata/repomd.xml
dnf repolist --all
dnf makecache --refresh
dnf repoquery --repo local-custom <representative-package>
```

若还需要包组或模块，应额外执行 `dnf group list` 或 `dnf module list` 验证对应元数据，不用普通包查询替代。

[Cheatsheet] 先找 `repomd.xml`；普通 RPM 目录用 `createrepo_c`；内容变化后 `--update`；客户端 `file:///` 绝对路径；本地仍需权限和 GPG；普通 `repodata` 不保证 `group/module` 元数据。

[诊断专题] 从症状定位配置、元数据、TLS、GPG、过滤或依赖层

仓库故障通常会在同一条 DNF 命令中连续暴露多个层次。诊断时不要一次修改多个开关，而应遵循：

症状

- 当前证据
- 假设
- 下一条最有区分度的证据
- 最小修复
- 再验证

① [诊断] 仓库没有出现在 `repolist --all`

症状：新建 repo 文件后，目标 ID 完全不存在。当前证据：

```
ls -l /etc/yum.repos.d/  
sed -n '1,160p' /etc/yum.repos.d/exam.repo  
dnf repolist --all
```

假设：文件扩展名错误、INI 语法错误、section 缺失、ID 重复或文件不可读。下一条证据：读取 DNF 的完整错误与配置文件；必要时使用 `dnf config-manager --dump <id>`，前提是插件存在。最小修复：修正文件名、section 或语法，不删除其他仓库。再验证：`dnf repolist --all`。

② [诊断] `repomd.xml` 404

症状：DNF 报错 URL 以 `/repodata/repomd.xml` 结尾并返回 404。假设：`baseurl` 指向了仓库上层、下层或错误产品路径。最有区分度证据：对照题目 URL 与服务器目录结构；读取错误中的完整最终 URL。最小修复：让 `baseurl` 指向真正包含 `repodata` 的根。再验证：`dnf makecache --refresh`，随后 `repoquery --repo`。边界：`dnf clean all` 不会修复服务器上的 404。

③ [诊断] 名称解析或连接失败

症状：`could not resolve host`、`connection timed out`、`network unreachable`。证据链：

```
getent hosts content.example.com  
ip route get <resolved-ip>  
curl -I http://content.example.com/path/
```

网络配置和 DNS 主归属第 18、19 章。本章只要求把错误定位到解析、路由、代理或服务端连接层，并保留 repo 配置中已经正确的部分。

④ [诊断] HTTPS 证书验证失败

症状：`certificate verify failed`、`unknown CA`、`certificate not yet valid/expired`。假设：系统时间错误、CA 不受信任、服务器证书链不完整或访问了错误主机名。证据：`timedatectl`、完整证书错误、系统 CA 配置和 URL 主机名。最小修复：修正时间、URL 或 CA 信任。禁止默认方案：`sslverify=0`。

⑤ [诊断] GPG key 或包签名失败

症状： public key not installed、GPG check FAILED、签名与 key 不匹配。当前证据： repo 的 `gpgcheck/gpgkey`、key 文件是否存在、完整事务错误。假设： key 路径错误、仓库包由另一把 key 签名、内容损坏或来源混用。最小修复： 使用题目或仓库官方提供的正确 key，修正来源；必要时清除失败下载后重试。再验证： 保持 `gpgcheck=1` 重新执行事务。边界： 导入了某把 key 不代表它一定匹配当前仓库。

⑥ [诊断] 元数据校验失败或缓存异常

症状： checksum mismatch、repomd 与数据不一致、使用旧候选。假设： 镜像正在同步、代理缓存不一致、本地缓存对应旧仓库定义。证据： 错误中的元数据文件和 checksum；repo URL 是否近期改变。最小修复： 先确认服务器端一致，再按需：

```
dnf clean metadata
dnf makecache --refresh
```

不要把本地清缓存当成服务器元数据损坏的修复。

⑦ [诊断] 仓库可用但包没有匹配项

证据链：

```
dnf repoquery --repo <repoid> <package>
dnf repoquery --available <package>
dnf module provides '<package-or-path>'
dnf module list
grep -R '^[:space:]]*\(\exclude\|excludepkgs\|includepkgs\)' /etc/dnf /etc/yum.repos.d
2>/dev/null
```

依次区分：包名错误、仓库内容缺失、架构不符、exclude/include、模块过滤和缓存问题。

⑧ [诊断] 依赖冲突或包被替换

症状： conflicting requests、nothing provides、cannot install the best candidate。当前证据： 完整 DNF 错误和 Transaction Summary。下一条证据：

```
dnf repoquery --requires <package>
dnf repoquery --whatprovides '<capability>'
dnf module list
dnf repolist --all
```

最小修复： 启用缺失的正确仓库、纠正 stream、移除错误 exclude，或选择题目要求的合法候选。边界： `--allowerase` 允许求解器删除冲突包，可能破坏其他任务；必须先理解它准备删除什么，不作为第一尝试。

⑨ [诊断] 本地仓库有 RPM 但 DNF 看不到

```
目录中是否有 repodata/repomd.xml
→ baseurl 是否使用正确的 file:/// 绝对路径
→ 目录和父目录是否可遍历
→ 元数据是否在添加 RPM 后更新
→ 客户端是否刷新缓存
→ 包架构、排除和模块规则是否允许候选
```

若 `repoquery` 能看到普通包但 `module list` 没有预期流，重点检查模块元数据是否随内容保留。

⑩ [诊断] 包组不存在或内容不完整

先确认启用仓库是否包含 `group/comps` 元数据：

```
dnf group list --hidden
dnf group info '<group>'
```

普通包查询成功不能证明组元数据存在。自建仓库若只运行 `createrepo_c` 而没有导入 `comps`，组列表为空可能是预期结果。

[Cheatsheet] repo 不见查文件和 ID；404 查仓库根；解析/连接错交给 DNS/网络层；TLS 查时间与 CA；GPG 查 key/来源；无包查 repo/架构/exclude/module；依赖错读完整摘要，不默认 allowerasing；本地仓库查 repodata、权限和高级元数据。

[操作专题] 工作迁移：最小变更、审计与自动化接口

真实服务器往往已经存在订阅仓库、自定义镜像、历史模块状态和多次软件事务。工作迁移的重点是让每次变更可解释、可回顾，并为后续 Ansible 自动化提供稳定目标，而不是在手工阶段制造隐含状态。

① [操作] 变更前保存软件来源基线

```
dnf repolist --all
dnf module list --enabled
dnf module list --installed
dnf history | head -20
```

必要时备份将修改的单个 repo 文件，而不是打包全部缓存。基线应能回答：新增了哪个 repo、改变了哪个 stream、事务前后差异是什么。

② [操作] 只修改与目标相关的字段

题目要求新增两个仓库时，新建一个清晰的 `.repo` 文件通常比编辑系统所有仓库更安全。已有仓库若与任务无冲突，不删除、不重命名。临时限定来源用命令选项，持久状态仍由 repo 文件表达。

③ [验证] 保留来源和事务证据

一个可审计的交付至少记录：

repo ID 和来源 URL
元数据刷新结果
目标包来自哪个 repoid
事务 ID 和摘要
跨章的已安装与功能验证入口

不要复制包含凭据的私有 URL、代理密码或订阅 token 到公开报告。

④ [接口] 为 RHCE 自动化准备声明式目标

后续 RHCE 章节可能使用 `ansible.builtin.yum_repository` 或 `ansible.builtin.dnf` 表达相同目标。手工阶段应先明确：

仓库 ID、baseurl、enabled、gpgcheck、gpgkey
目标包或包组
模块 stream/profile
验证命令和预期状态

本章不展开 Playbook，但避免只留下“执行过某条命令”的过程描述。

⑤ [边界] 不把内部仓库访问凭据写入普通 repo 模板

若仓库需要用户名、密码、客户端证书或代理凭据，应按组织的凭据管理方案处理。考试通常提供无需秘密的固定 URL；真实工作中不应把密码直接提交到公共 Git 仓库或 Anki 卡片。

[Cheatsheet] 先保存 repo/module/history 基线；最小修改单个配置；记录来源、候选和事务；临时覆盖与持久配置分开；为自动化保留目标字段，不泄露凭据。

[经典任务] 配置带 GPG 校验的 BaseOS/AppStream 仓库并安装软件包

环境

系统为 RHEL 9。当前 `/etc/yum.repos.d/` 中没有以下两个仓库。题目提供的内容服务器和 key 在考试环境中应当可用，但本章没有连接真实 VM。

项目	参数
BaseOS repo ID	<code>exam-baseos</code>
BaseOS URL	<code>http://content.example.com/rhel9.0/x86_64/dvd/BaseOS</code>
AppStream repo ID	<code>exam-appstream</code>

项目	参数
AppStream URL	<code>http://content.example.com/rhel9.0/x86_64/dvd/AppStream</code>
GPG key	<code>file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release</code>
目标包	<code>httpd</code>

当前状态

- 其他系统仓库可能存在，不允许无差别删除；
- `httpd` 是否已安装未知；
- 不假设任何模块流已被修改；
- 本任务只验收到软件包和 DNF 事务层，不要求配置或启动 Web 服务。

目标终态

1. 两个仓库持久定义并默认启用；
2. `gpgcheck=1`，使用题目给出的 key；
3. 两个仓库元数据能够刷新；
4. `bash` 可从 `exam-baseos` 查询；
5. `httpd` 可从 `exam-appstream` 查询；
6. 安装 `httpd`，并能从 history 找到对应事务；
7. 不改变无关模块流和包组状态。

限制条件

- 不设置 `gpgcheck=0` 或 `sslvcrify=0`；
- 不使用 `rpm --nodeps`、`rpm --force`；
- 不默认使用 `--allowerasing`；
- 不删除系统中其他有效仓库；
- 不把包安装成功写成 Web 服务已经可用。

验收矩阵

验收层	推荐证据
配置	读取 <code>/etc/yum.repos.d/exam.repo</code>
识别/启用	<code>dnf repolist --all</code>
元数据	<code>dnf makecache --refresh</code>
BaseOS 内容	<code>dnf repoquery --repo exam-baseos bash</code>
AppStream 内容	<code>dnf repoquery --repo exam-appstream httpd</code>

验收层	推荐证据
事务预览	<code>dnf install --assumeno httpd</code>
安装状态	<code>rpm -q httpd</code> （第 16 章接口）
审计	<code>dnf history</code> 、 <code>dnf history info <ID></code>

请先独立完成。参考解答从下一页开始。

[参考解答] 从 repo 文件推进到可审计事务

① 调查并保存基线

```
ls -l /etc/yum.repos.d/  
dnf repolist --all  
dnf module list --enabled  
rpm -q httpd
```

`rpm -q` 只是确认初始已安装状态，完整 RPM 语义归第 16 章。不要因为目标包已经安装就跳过仓库验证：题目仍要求两个仓库定义和内容证据。

② 创建最小仓库配置

```
cat > /etc/yum.repos.d/exam.repo <<'EOF'  
[exam-baseos]  
name=Exam Base0S  
baseurl=http://content.example.com/rhel9.0/x86_64/dvd/Base0S  
enabled=1  
gpgcheck=1  
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release  
  
[exam-appstream]  
name=Exam AppStream  
baseurl=http://content.example.com/rhel9.0/x86_64/dvd/AppStream  
enabled=1  
gpgcheck=1  
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release  
EOF
```

这里使用单引号 here-document，便于未来包含 DNF 变量时避免 Shell 提前展开。配置中没有凭据或无关字段。

③ 验证解析与元数据

```
dnf repolist --all
dnf makecache --refresh
```

若失败，不继续安装。先根据完整错误定位：

- 仓库 ID 不出现：repo 文件/section 问题；
- `repomd.xml` 404：URL 不是仓库根；
- 无法解析主机：DNS 层；
- 证书失败：时间/CA/TLS 层；
- checksum 错误：缓存或服务端同步层。

④ 限定仓库查询代表包

```
dnf repoquery --repo exam-baseos bash
dnf repoquery --repo exam-appstream httpd
```

这两条分别证明仓库提供代表内容。若 `httpd` 无结果，继续检查 AppStream URL、架构、exclude 和 module filtering，而不是关闭 GPG。

⑤ 预览并提交事务

```
dnf install --assumeno httpd
dnf install httpd
```

预览时确认没有意外删除、降级或跨来源替换。题目没有要求只使用这两个仓库，因此不必主动禁用所有其他仓库；若评分明确要求限定来源，可使用：

```
dnf --disablerepo='*' \
  --enablerepo=exam-baseos,exam-appstream \
  install httpd
```

必须同时启用 BaseOS 与 AppStream，以保留依赖解析范围。

⑥ 分层验证

```
rpm -q httpd
dnf history
# 根据实际列表选择本次事务 ID:
dnf history info <TRANSACTION_ID>
```

静态预期：

- `rpm -q` 应确认包在已安装数据库中；
- `history info` 应显示本次安装涉及的软件包和动作；
- 本任务到此只完成软件层，不声称 `httpd.service` `active/enabled` 或 HTTP 可访问。

⑦ 典型错误

- 只执行 `repolist`，没有实际刷新元数据；
- `baseurl` 指向 `.../dvd` 而不是 `.../dvd/BaseOS` 或 `AppStream`；
- 为消除签名报错设置 `gpgcheck=0`；
- 只启用 `AppStream`，导致 `BaseOS` 依赖不可用；
- 未读事务摘要直接接受 `--allowerase`；
- 安装成功后扩大为“Web 服务已完成”。

[经典任务] 诊断 “仓库可见但包不可解析或签名失败”

环境和症状

系统存在仓库 `training-appstream`：

```
[training-appstream]
name=Training AppStream
baseurl=https://repo.example.com/rhel9/AppStream/Packages
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-training
```

已知：

```
dnf repolist --all 能看到 training-appstream 为 enabled
但 dnf makecache --refresh 下载 repodata/repomd.xml 时返回 404
```

修复 URL 后，元数据刷新成功；执行 `dnf install example-server` 又报告 GPG key 不匹配。题目要求保持 TLS 和 GPG 校验。

目标终态

1. 找到真正的仓库根并修正 `baseurl`；
2. 元数据刷新成功；
3. `example-server` 能从该仓库查询；
4. 使用与仓库包签名匹配的 key，保持 `gpgcheck=1`；
5. 安装前读事务摘要；
6. 形成每次判断的证据，不通过关闭保护绕过问题。

限制条件

- 不设置 `sslverify=0`；
- 不设置 `gpgcheck=0`；
- 不把 `dnf clean all` 当成错误 URL 的修复；
- 不默认使用 `--allow-erase`；
- 不修改无关仓库。

验收证据

```
完整失败 URL
修正后的 repo 文件
makecache --refresh
repoquery --repo training-appstream example-server
key 文件和 gpgkey 对应关系
保持 gpgcheck=1 的成功事务或明确的下一步证据
```

请先独立完成。参考解答从下一页开始。

[参考解答] 用最有区分度的证据逐层修复

① 读取完整错误 URL

错误 URL 若为：

```
https://repo.example.com/rhel9/AppStream/Packages/repodata/repomd.xml
```

而服务端的 `repodata` 实际位于 `.../AppStream/repodata/`，则 `Packages` 是错误的下层目录。最小修复是：

```
baseurl=https://repo.example.com/rhel9/AppStream
```

不要先清缓存，因为 404 已经证明请求抵达服务器，只是路径不存在。

② 再验证元数据和候选

```
dnf makecache --refresh
dnf repoquery --repo training-appstream example-server
```

若元数据成功但包无结果，按以下顺序检查：

包名与架构

- 该 repo 是否实际包含包
- exclude/include
- module filtering

可以补充：

```
dnf module provides 'example-server'  
dnf module list
```

③ 调查 GPG 失败

```
grep -E '^(gpgcheck|gpgkey)= ' /etc/yum.repos.d/training.repo  
ls -l /etc/pki/rpm-gpg/RPM-GPG-KEY-training  
timedatectl
```

读取完整事务错误，确认是“key 尚未导入”“签名者与 key 不匹配”还是包本身损坏。系统时间虽然更常影响 TLS，但明显错误时间也会破坏整个信任调查，因此只作为证据核对，不用来替代签名者检查。

④ 最小修复 key 或仓库来源

使用题目/仓库发布者提供的正确公钥，修正 `gpgkey=`；如果包来自错误镜像，应修正仓库而不是导入未知 key。保持：

```
gpgcheck=1
```

然后预览事务：

```
dnf install --assumeno example-server
```

确认来源仓库、安装和删除集合合理后再提交。

⑤ 再验证并记录证据

```
dnf install example-server  
rpm -q example-server  
dnf history  
dnf history info <TRANSACTION_ID>
```

本章未在真实 RHEL 9 VM 执行，因此不提供虚构输出。实际环境若仍失败，应保留完整错误并继续在当前层调查，而不是打开绕过选项。

⑥ 典型错误分支

错误做法	为什么不成立
<code>dnf clean all</code> 后反复重试 404	删除本地缓存不能创建服务器端不存在的路径
<code>sslverify=0</code>	404 已经不是证书问题；关闭 TLS 验证还移除安全属性
<code>gpgcheck=0</code>	掩盖 key/来源错误，不满足题目终态
导入任意搜索到的 key	无法证明该 key 是仓库发布者的受信任 key
<code>--allowerasing</code>	GPG 错误不是依赖删除问题

[本章收束] 从软件来源推进到可审计的事务终态

DNF 的核心不是一组安装命令，而是一条来源和解析链。repo 文件决定默认可见的内容来源和信任策略；元数据决定客户端能够认识哪些包、模块和组；优先级、排除和模块状态进一步塑造候选；求解器把候选组合成事务；history 和 RPM 层证据再说明实际发生了什么。

以后面对软件问题，应先问：

我正在证明配置、元数据、候选、事务，还是最终功能？

仓库不见时查定义；`repomd.xml` 错时查仓库根和传输；包不见时查仓库范围、架构、排除和模块；GPG 失败时查 key 与来源；依赖冲突时读完整摘要和依赖，不默认删除现有包。只有每一层都由相应证据支撑，软件来源和事务才真正可解释、可复核、可迁移。

工作方法：先判定失败层，再选择最小动作

面对新的 DNF 题目或真实故障，优先把现象改写成一个可判定问题：是客户端没有加载仓库定义，还是元数据无法取得；是候选包被过滤，还是依赖求解失败；是包签名不可信，还是事务完成后还缺少服务配置。每次只改变能够解释当前证据的最小对象，并在同一层重新验证。

- 先读完整错误
- 标记失败层
- 选择最有区分度的证据
- 做最小修复
- 在原层复验
- 再进入下一层验收

主要判断表

看到的证据	可以确认	仍不能确认	下一步
repo ID 出现在 <code>dnf repolist --all</code>	配置被 DNF 识别，启用状态可见	URL 与元数据可用	<code>dnf makecache --refresh</code>
<code>makecache --refresh</code> 成功	启用仓库的元数据可取得并通过当前校验	目标包一定可见	限定仓库执行 <code>repoquery</code>
<code>repoquery</code> 返回目标候选	当前元数据和过滤规则允许看到该构建	依赖一定可解、事务已经提交	<code>dnf install --assumeno</code>
Transaction Summary 合理	求解器生成了预期事务计划	事务已经发生	明确确认后提交
<code>dnf history info</code> 记录成功事务	DNF 事务层发生了可审计变化	应用配置和服务功能正确	转交对应配置、systemd 和功能验证
模块 stream 已启用	候选版本线发生改变	profile 已安装	查询模块状态与已安装 profile
包组安装完成	一组软件包按组元数据完成事务	组内服务已经配置并运行	转交相应服务章节

本章离场检查

进入下一章前，应能够在不依赖“反复试命令”的情况下完成以下判断：

- 1. 看到一个 repo 文件时，能够区分文件名、section ID、显示名和内容 URL；
- 2. 能够说明 `repolist`、`makecache --refresh`、`repoquery` 和事务摘要分别证明哪一层；
- 3. 遇到“仓库有包但客户端找不到”时，能够继续检查架构、exclude、priority 和 module filtering；
- 4. 能够区分 TLS 连接问题、GPG key 问题和包签名问题，而不是关闭验证；
- 5. 能够区分模块 stream、profile、包组和普通包事务；
- 6. 能够从 DNF history 交接到 RPM、systemd、网络和功能层验证。

配置可见 → 元数据可用 → 候选可解析 → 来源可信 → 事务成功 → 功能交接

向下一章交接

本章结束在“软件来源、候选集合和事务证据”这一层。第 18 章《NetworkManager、IPv4 地址与路由》将完整处理网络连接、地址和路由；第 19 章《主机名、NSS 与 DNS 名称解析》将完整处理名称解析。DNF 诊断中出现 DNS、路由或连接错误时，本章只负责识别错误层并保留证据，不在此处重复展开网络配置。RPM 已安装数据库、文件归属和校验继续由第 16 章负责。